FoodLoop - API Documentation

---

1. Overview

FoodLoop exposes a RESTful API consumed by the mobile application, merchant dashboard, and admin panel.

All API endpoints return JSON responses and use HTTPS.

Authentication is handled using JWT Bearer Tokens.

---

2. Authentication

Most endpoints require authentication.

```
Authorization: Bearer <access_token>
```

Public endpoints (login, registration, product browsing) do not require authentication unless specified.

---

3. API Modules

Module	Description
Authentication	User authentication and authorization
Users	User profiles and preferences
Stores	Merchant and store management
Marketplace	Product listings and browsing
Orders	Purchase and order management
Payments	Payment processing
AI	Product recognition and OCR
Reviews	Ratings and feedback
Notifications	User notifications
Support	Customer support
Admin	Platform administration

---

4. Authentication APIs

Register

`POST /auth/register`

Creates a new user account.

Request

```
{
  "name": "",
  "email": "",
  "password": ""
}
```

Response

```
{
  "user": {},
  "accessToken": "",
  "refreshToken": ""
}
```

Login

`POST /auth/login`

Returns access and refresh tokens.

Refresh Token

`POST /auth/refresh`

Returns a new access token.

Logout

`POST /auth/logout`

Invalidates the current session.

Forgot Password

`POST /auth/forgot-password`

Sends password reset email.

Reset Password

`POST /auth/reset-password`

Updates the user's password.

---

5. User APIs

Get Current User

`GET /users/me`

Returns authenticated user information.

Update Profile

`PATCH /users/me`

Updates profile information.

Examples:

- Name
- Profile picture
- Preferred language

Address Management

```
GET    /users/me/addresses
POST   /users/me/addresses
PATCH  /users/me/addresses/{id}
DELETE /users/me/addresses/{id}
```

Notification Preferences

`PATCH /users/me/preferences`

Updates notification and application settings.

---

6. Store APIs

List Stores

`GET /stores`

Supports filtering and sorting by:

- Category
- Rating
- Distance
- Verification status

Store Details

`GET /stores/{id}`

Returns:

- Store profile
- Rating
- Product listings
- Opening hours

Create Store

`POST /stores`

Merchant only.

Update Store

`PATCH /stores/{id}`

Merchant only.

---

7. Marketplace APIs

Browse Listings

`GET /listings`

Supports:

- Search
- Category
- Price range
- Nearby stores
- Expiration date
- Availability

Listing Details

`GET /listings/{id}`

Returns:

- Product information
- Images
- Store details
- Remaining quantity

Create Listing

`POST /listings`

Merchant only.

Update Listing

`PATCH /listings/{id}`

Merchant only.

Delete Listing

`DELETE /listings/{id}`

Merchant only.

Upload Product Images

`POST /listings/{id}/images`

Favorite Product

`POST /listings/{id}/favorite`

Remove Favorite

`DELETE /listings/{id}/favorite`

---

8. Order APIs

Create Order

`POST /orders`

Creates a new order from one or more listings.

Order History

`GET /orders`

Returns all previous orders.

Order Details

`GET /orders/{id}`

Returns:

- Items
- Payment
- Merchant
- Status

Cancel Order

`PATCH /orders/{id}/cancel`

Available while the order has not yet been prepared.

Merchant Order Queue

`GET /merchant/orders`

Merchant dashboard endpoint.

Update Order Status

`PATCH /merchant/orders/{id}`

Possible statuses:

- Confirmed
- Preparing
- Ready for Pickup
- Completed
- Cancelled

---

9. Payment APIs

Create Payment

`POST /payments`

Initiates payment for an order.

Payment Status

`GET /payments/{id}`

Returns payment information.

Refund Request

`POST /payments/{id}/refund`

Creates a refund request for review.

---

10. AI APIs

Upload Product Image

`POST /ai/product-recognition`

Uploads an image for AI analysis.

Response may include:

- Product name
- Category
- OCR results
- Expiration date
- Confidence score

Validate AI Result

`POST /ai/product-recognition/confirm`

Allows the merchant to confirm or edit AI-generated information before creating the listing.

---

11. Review APIs

Create Review

`POST /reviews`

Requires a completed order.

Store Reviews

`GET /stores/{id}/reviews`

Returns reviews for a specific store.

Update Review

`PATCH /reviews/{id}`

Users may edit their own review within a configurable time window.

---

12. Notification APIs

Get Notifications

`GET /notifications`

Returns all notifications for the authenticated user.

Mark as Read

`PATCH /notifications/{id}`

Mark All as Read

`PATCH /notifications/read-all`

---

13. Support APIs

Create Ticket

`POST /support/tickets`

Creates a support request.

List Tickets

`GET /support/tickets`

Returns all tickets created by the current user.

Ticket Details

`GET /support/tickets/{id}`

Reply to Ticket

`POST /support/tickets/{id}/messages`

Adds a message or attachment to an existing ticket.

Close Ticket

`PATCH /support/tickets/{id}/close`

---

14. Admin APIs

Restricted to administrators.

Users

```
GET /admin/users
PATCH /admin/users/{id}
DELETE /admin/users/{id}
```

Stores

```
GET /admin/stores
PATCH /admin/stores/{id}
```

Includes merchant verification and moderation actions.

Listings

```
GET /admin/listings
PATCH /admin/listings/{id}
DELETE /admin/listings/{id}
```

Used to moderate reported or inappropriate listings.

Reports

```
GET /admin/reports
PATCH /admin/reports/{id}
```

Handles user and content reports.

Support

`GET /admin/support`

Allows administrators to oversee support tickets and escalations.

---

15. Standard Response Format

Successful responses follow a consistent structure:

```
{
  "success": true,
  "data": {}
}
```

Error responses:

```
{
  "success": false,
  "message": "Resource not found.",
  "errors": []
}
```

---

16. HTTP Status Codes

Code	Description
200	Success
201	Resource Created
204	No Content
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Validation Error
429	Too Many Requests
500	Internal Server Error

17. API Security

The API follows several security practices:

- JWT-based authentication
- Role-Based Access Control (RBAC)
- Password hashing
- Request validation
- HTTPS-only communication
- Rate limiting on sensitive endpoints
- Input sanitization
- File upload validation
- Audit logging for administrative actions